
Fuzzing Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *October 22nd, 2019*

Setup

1. The lab target ships in `fuzzing/targets/lab\_target.c`: a deliberately vulnerable checksum-gated parser with the wire format [magic “CKS”] [2-byte `payload_len`] [2-byte checksum = sum of payload bytes mod 2^{16}] [payload]. The three validation gates are exactly the “staircase of friction” from the Mutation Fuzzing lecture: dumb mutation rarely satisfies all three.

2. Build the ASan-instrumented binary and the standalone driver (both verified on the reference host):

```
1 clang -g -O1 -fsanitize=fuzzer-no-link,address \\  
2     -c lab_target.c -o lab_target.o  
3 clang -g -O1 -fsanitize=address \\  
4     lab_target.o harness.c main.c -o lab_target_asan
```

3. If your clang ships the libFuzzer runtime (`-fsanitize=fuzzer` linking works), also build the true fuzzer:

```
1 clang -g -O1 -fsanitize=fuzzer,address lab_target.o harness.c -o lab_fuzz
```

Baseline: Learn the Target by Hand

1. Run the provided seeds through the ASan binary and record exit statuses. Verified reference behavior (exit 0 = accepted, 2 = validation-rejected):

Seed	Content	Exit
seed_ok	valid CKS, 30-byte payload, best sum	0
seed_badmagic	magic byte 0 corrupted	2 (gate 1)
seed_badsum	checksum field = 0xBEEF, wrong for payload	2 (gate 3)
seed_shift	type byte 0x80 (negative as char)	2 *
seed_over	<code>payload_len = 300</code> into 64-byte stack buf	crash: ASan

2. * `seed_shift` exits 2 on the reference build because the bug in the type gate (char vs unsigned comparison) compiles to a benign path under this optimizer — a live example of the lecture’s point that sanitizer/tool visibility depends on the build. How would you make that harness path **fail loudly** instead of silently? Try `-O0` and `-fsanitize=undefined`.
3. Which gate does `seed\_badmagic` fail? Why does `seed\_len0` (empty payload) not crash?

Exercise 1 — Dumb Mutation (black box)

1. Using only `seed\_ok` and any bit-flip strategy you like (write 20 lines of Python, or use `radamsa` if installed), attempt to produce crashes.
2. Track two numbers over your mutations: (a) how many produced a *checksum that passed gate 3*, (b) how many crashed.
3. Expected result (and the lesson): crashes require passing gates 1–3 simultaneously; random bit flips essentially never satisfy the checksum while also inflating `payload_len`. Quantify it — how many trials per crash?

Exercise 2 — Coverage-Guided Feedback (if `lab_fuzzer` built)

1. Make a corpus directory containing only `seed\_ok`; run:

```
1 ./lab_fuzzer -max_len=4096 corpus/
```
2. Watch coverage growth. Compare wall-time-to-first-crash against your Exercise 1 count. Explain in one sentence why feedback closes the checksum stair faster than mutation.
3. Add `seed\_shift` to the corpus and rerun: does the fuzzer find the sign-comparison bug path that the ASan build silenced? What does this tell you about seed curation?

Exercise 3 — Crash Triage (the lecture’s pipeline, in practice)

1. Take the crashing input(s) found in Exercise 1/2 and run the full **Crash Triage and Exploitability** pipeline from the lecture:
 - (a) Dedupe: if you have multiple crashing inputs, group by stack hash. Do they all show `lab\_parse` at frame #0, `lab\_target.c:46`?
 - (b) Minimize: shrink `seed\_over`-style inputs to the smallest payload that still crashes. What is the minimum `payload_len` that trips ASan’s redzone, and why exactly 65? (Allocation is 64; the first OOB byte lands in the redzone.)
 - (c) Reproduce deterministically: run the minimized input three times; confirm identical stack each time.
 - (d) Root cause: express the bug in two sentences naming the missing check and the wrong buffer size.
 - (e) Exploitability: price the primitive — attacker controls length and content of the overflow; the buffer is a 64-byte stack local adjacent to (what does `otool -tv` show is saved above it?). Apply the lecture’s mitigation map: with ASan this input dies at the first OOB byte; without ASan, where does control-flow-relevant overflow begin?
2. Deliverable: one-paragraph report per the source auditing lecture’s **From Bug to Report** format, with your minimized reproducer attached.

Exercise 4 — Structure-Aware Generative Fuzzing (stretch)

1. Write a 30-line generator (Python direct construction works) emitting CKS frames with (a) correct checksums and payload lengths sweeping 1..4096, (b) checksums off-by-one, (c) claimed lengths lying 1–8 bytes high versus actual availability.
2. Which of the three generator modes reaches the memcpy? Which discovers the gate-2 boundary (`payload\_len > size - 8`) failure when the container is also fuzzed?
3. Explain, citing the lecture's Generative Fuzzing weaknesses, why a too-faithful generator finds nothing: the model and the parser agree exactly, so no implementation assumption is violated.

Deliverables

1. Mutation-vs-feedback wall-time comparison table (Exercises 1 vs 2).
2. Minimized crash reproducer and one-paragraph exploitability-priced report (Exercise 3).
3. Generator source plus reach analysis (Exercise 4).